



华南理工大学
South China University of Technology

操作系统实验报告

郑重声明：

- 1、实验手册中的所有实验均有本人独立编码、调试和测试。
- 2、实验手册中给出的实验数据和结果完全由本人所完成的程序给出。
- 3、本人了解：不按照前两条要求所完成的实验报告已经构成了抄袭或造假行为，本人将承担相应的不良后果。

姓名：_____（签名）

学号：_____

班级：_____ 22 计算机科学与技术 1 班

提交日期：_____ 2025/06/08

成绩：_____

一、实验目的

本实验要求编写一个简单的进程调度程序以加深理解有关进程控制块、进程队列的概念，并体会进程调度算法的具体实施办法。

二、实验内容

选择任意一种算法设计一个有 N （不少于 4）个进程共行的进程调度程序。

原理分析：

- (1) 假设系统有 5 个进程，每个进程用一个进程控制块来代表。
- (2) 为每个进程任意确定一个要求运行时间和到达时间。
- (3) 按照进程到达的先后顺序拍成一个循环队列，再设一个队首指针指向第一个到达进程的首地址
- (4) 执行处理机调度时，开始选择队首的第一个程序运行。另外，再设一个当前运行进程的指针，指向当前正在运行的进程。
- (5) 进程运行一次后，估计运行时间减一，输出当前运行进程的名字
- (6) 进程运行一次后，调整当前运行指针指向该进程的连接指针所指进程，已指示应运行进程。同时还应判断给该进程的剩余时间是否为 0，若不为 0 则等待下一轮的运行，否则将该进程的状态置为完成态，并推出循环队列。
- (7) 若就绪队列不空，则重复 (5) 和 (6)，直至所有进程运行完为止。
- (8) 在所设计的程序中，应包含显示或打印语句。显示或打印每次选中的进程的名称及运行一次后队列的变化情况

三、实验环境

编译标准：C++ 17

四、实验步骤（附上截图）

在命令行中，输入以下参数，得到模拟调度结果。

请输入进程数量 ($N \geq 4$): 4

进程 1 名称: P1

进程 1 到达时间: 0

进程 1 要求运行时间: 20

进程 2 名称: P2

进程 2 到达时间: 5

进程 2 要求运行时间: 5

进程 3 名称: P3

进程 3 到达时间: 10

进程 3 要求运行时间: 16

进程 4 名称: P4

进程 4 到达时间: 20

进程 4 要求运行时间: 20

(请放大观看)

Round Robin 算法:

```
使用 Round Robin 调度算法
Time 0: 运行进程名 P1 (剩余 20->19)
        队列: P1(剩余 19)

Time 1: 运行进程名 P1 (剩余 19->18)
        队列: P1(剩余 18)

Time 2: 运行进程名 P1 (剩余 18->17)
        队列: P1(剩余 17)

Time 3: 运行进程名 P1 (剩余 17->16)
        队列: P1(剩余 16)

Time 4: 运行进程名 P1 (剩余 16->15)
        队列: P2(剩余 5)    P1(剩余 15)

Time 5: 运行进程名 P2 (剩余 5->4)
        队列: P1(剩余 15)    P2(剩余 4)

Time 6: 运行进程名 P1 (剩余 15->14)
        队列: P2(剩余 4)    P1(剩余 14)

Time 7: 运行进程名 P2 (剩余 4->3)
        队列: P1(剩余 14)    P2(剩余 3)

Time 8: 运行进程名 P1 (剩余 14->13)
        队列: P2(剩余 3)    P1(剩余 13)

Time 9: 运行进程名 P2 (剩余 3->2)
        队列: P1(剩余 13)    P3(剩余 16) P2(剩余 2)

Time 10: 运行进程名 P1 (剩余 13->12)
        队列: P3(剩余 16)    P2(剩余 2)    P1(剩余 12)

Time 11: 运行进程名 P3 (剩余 16->15)
        队列: P2(剩余 2)    P1(剩余 12) P3(剩余 15)

Time 12: 运行进程名 P2 (剩余 2->1)
        队列: P1(剩余 12)    P3(剩余 15) P2(剩余 1)

Time 13: 运行进程名 P1 (剩余 12->11)
        队列: P3(剩余 15)    P2(剩余 1)    P1(剩余 11)

Time 14: 运行进程名 P3 (剩余 15->14)
        队列: P2(剩余 1)    P1(剩余 11) P3(剩余 14)

Time 15: 运行进程名 P2 (剩余 1->0)  <进程 P2 已完成, 踢出队列>
        队列: P1(剩余 11)    P3(剩余 14)

Time 16: 运行进程名 P1 (剩余 11->10)
        队列: P3(剩余 14)    P1(剩余 10)

Time 17: 运行进程名 P3 (剩余 14->13)
        队列: P1(剩余 10)    P3(剩余 13)

Time 18: 运行进程名 P1 (剩余 10->9)
        队列: P3(剩余 13)    P1(剩余 9)

Time 19: 运行进程名 P3 (剩余 13->12)
        队列: P1(剩余 9)    P4(剩余 20) P3(剩余 12)

Time 20: 运行进程名 P1 (剩余 9->8)
        队列: P4(剩余 20)    P3(剩余 12) P1(剩余 8)

Time 21: 运行进程名 P4 (剩余 20->19)
        队列: P3(剩余 12)    P1(剩余 8)    P4(剩余 19)

Time 22: 运行进程名 P3 (剩余 12->11)
        队列: P1(剩余 8)    P4(剩余 19) P3(剩余 11)

Time 23: 运行进程名 P1 (剩余 8->7)
        队列: P4(剩余 19)    P3(剩余 11) P1(剩余 7)

Time 24: 运行进程名 P4 (剩余 19->18)
        队列: P3(剩余 11)    P1(剩余 7)    P4(剩余 18)

Time 25: 运行进程名 P3 (剩余 11->10)
        队列: P1(剩余 7)    P4(剩余 18) P3(剩余 10)

Time 26: 运行进程名 P1 (剩余 7->6)
        队列: P4(剩余 18)    P3(剩余 10) P1(剩余 6)

Time 27: 运行进程名 P4 (剩余 18->17)
        队列: P3(剩余 10)    P1(剩余 6)    P4(剩余 17)

Time 28: 运行进程名 P3 (剩余 10->9)
```

```
        队列: P1(剩余 6)    P4(剩余 17) P3(剩余 9)

Time 29: 运行进程名 P1 (剩余 6->5)
        队列: P4(剩余 17)    P3(剩余 9)    P1(剩余 5)

Time 30: 运行进程名 P4 (剩余 17->16)
        队列: P3(剩余 9)    P1(剩余 5)    P4(剩余 16)

Time 31: 运行进程名 P3 (剩余 9->8)
        队列: P1(剩余 5)    P4(剩余 16) P3(剩余 8)

Time 32: 运行进程名 P1 (剩余 5->4)
        队列: P4(剩余 16)    P3(剩余 8)    P1(剩余 4)

Time 33: 运行进程名 P4 (剩余 16->15)
        队列: P3(剩余 8)    P1(剩余 4)    P4(剩余 15)

Time 34: 运行进程名 P3 (剩余 8->7)
        队列: P1(剩余 4)    P4(剩余 15) P3(剩余 7)

Time 35: 运行进程名 P1 (剩余 4->3)
        队列: P4(剩余 15)    P3(剩余 7)    P1(剩余 3)

Time 36: 运行进程名 P4 (剩余 15->14)
        队列: P3(剩余 7)    P1(剩余 3)    P4(剩余 14)

Time 37: 运行进程名 P3 (剩余 7->6)
        队列: P1(剩余 3)    P4(剩余 14) P3(剩余 6)

Time 38: 运行进程名 P1 (剩余 3->2)
        队列: P4(剩余 14)    P3(剩余 6)    P1(剩余 2)

Time 39: 运行进程名 P4 (剩余 14->13)
        队列: P3(剩余 6)    P1(剩余 2)    P4(剩余 13)

Time 40: 运行进程名 P3 (剩余 6->5)
        队列: P1(剩余 2)    P4(剩余 13) P3(剩余 5)

Time 41: 运行进程名 P1 (剩余 2->1)
        队列: P4(剩余 13)    P3(剩余 5)    P1(剩余 1)

Time 42: 运行进程名 P4 (剩余 13->12)
        队列: P3(剩余 5)    P1(剩余 1)    P4(剩余 12)

Time 43: 运行进程名 P3 (剩余 5->4)
        队列: P1(剩余 1)    P4(剩余 12) P3(剩余 4)

Time 44: 运行进程名 P1 (剩余 1->0)  <进程 P1 已完成, 踢出队列>
        队列: P4(剩余 12)    P3(剩余 4)

Time 45: 运行进程名 P4 (剩余 12->11)
        队列: P3(剩余 4)    P4(剩余 11)

Time 46: 运行进程名 P3 (剩余 4->3)
        队列: P4(剩余 11)    P3(剩余 3)

Time 47: 运行进程名 P4 (剩余 11->10)
        队列: P3(剩余 3)    P4(剩余 10)

Time 48: 运行进程名 P3 (剩余 3->2)
        队列: P4(剩余 10)    P3(剩余 2)

Time 49: 运行进程名 P4 (剩余 10->9)
        队列: P3(剩余 2)    P4(剩余 9)

Time 50: 运行进程名 P3 (剩余 2->1)
        队列: P4(剩余 9)    P3(剩余 1)

Time 51: 运行进程名 P4 (剩余 9->8)
        队列: P3(剩余 1)    P4(剩余 8)

Time 52: 运行进程名 P3 (剩余 1->0)  <进程 P3 已完成, 踢出队列>
        队列: P4(剩余 8)

Time 53: 运行进程名 P4 (剩余 8->7)
        队列: P4(剩余 7)

Time 54: 运行进程名 P4 (剩余 7->6)
        队列: P4(剩余 6)

Time 55: 运行进程名 P4 (剩余 6->5)
        队列: P4(剩余 5)

Time 56: 运行进程名 P4 (剩余 5->4)
        队列: P4(剩余 4)

Time 57: 运行进程名 P4 (剩余 4->3)
        队列: P4(剩余 3)

Time 58: 运行进程名 P4 (剩余 3->2)
        队列: P4(剩余 2)

Time 59: 运行进程名 P4 (剩余 2->1)
        队列: P4(剩余 1)

Time 60: 运行进程名 P4 (剩余 1->0)  <进程 P4 已完成, 踢出队列>
        队列: 空

所有进程调度完成, 总时间 = 61

Process finished with exit code 0
```

（拓展）抢占式 SJF 算法：（请放大观看）

```
使用 抢占式短作业优先 调度算法
Time 0: 运行进程名 P1 (剩余 20->19)
        队列: P1(19)

Time 1: 运行进程名 P1 (剩余 19->18)
        队列: P1(18)

Time 2: 运行进程名 P1 (剩余 18->17)
        队列: P1(17)

Time 3: 运行进程名 P1 (剩余 17->16)
        队列: P1(16)

Time 4: 运行进程名 P1 (剩余 16->15)
        队列: P1(15)

Time 5: 运行进程名 P2 (剩余 5->4)
        队列: P1(15) P2(4)

Time 6: 运行进程名 P2 (剩余 4->3)
        队列: P1(15) P2(3)

Time 7: 运行进程名 P2 (剩余 3->2)
        队列: P1(15) P2(2)

Time 8: 运行进程名 P2 (剩余 2->1)
        队列: P1(15) P2(1)

Time 9: 运行进程名 P2 (剩余 1->0)  <进程 P2 已完成, 踢出队列>
        队列: P1(15)

Time 10: 运行进程名 P1 (剩余 15->14)
        队列: P1(14) P3(16)

Time 11: 运行进程名 P1 (剩余 14->13)
        队列: P1(13) P3(16)

Time 12: 运行进程名 P1 (剩余 13->12)
        队列: P1(12) P3(16)

Time 13: 运行进程名 P1 (剩余 12->11)
        队列: P1(11) P3(16)

Time 14: 运行进程名 P1 (剩余 11->10)
        队列: P1(10) P3(16)

Time 15: 运行进程名 P1 (剩余 10->9)
        队列: P1(9) P3(16)

Time 16: 运行进程名 P1 (剩余 9->8)
        队列: P1(8) P3(16)

Time 17: 运行进程名 P1 (剩余 8->7)
        队列: P1(7) P3(16)

Time 18: 运行进程名 P1 (剩余 7->6)
        队列: P1(6) P3(16)

Time 19: 运行进程名 P1 (剩余 6->5)
        队列: P1(5) P3(16)

Time 20: 运行进程名 P1 (剩余 5->4)
        队列: P1(4) P3(16) P4(20)

Time 21: 运行进程名 P1 (剩余 4->3)
        队列: P1(3) P3(16) P4(20)

Time 22: 运行进程名 P1 (剩余 3->2)
        队列: P1(2) P3(16) P4(20)

Time 23: 运行进程名 P1 (剩余 2->1)
        队列: P1(1) P3(16) P4(20)

Time 24: 运行进程名 P1 (剩余 1->0)  <进程 P1 已完成, 踢出队列>
        队列: P3(16) P4(20)

Time 25: 运行进程名 P3 (剩余 16->15)
        队列: P3(15) P4(20)

Time 26: 运行进程名 P3 (剩余 15->14)
        队列: P3(14) P4(20)

Time 27: 运行进程名 P3 (剩余 14->13)
        队列: P3(13) P4(20)

Time 28: 运行进程名 P3 (剩余 13->12)
        队列: P3(12) P4(20)

Time 29: 运行进程名 P3 (剩余 12->11)
        队列: P3(11) P4(20)

Time 30: 运行进程名 P3 (剩余 11->10)
        队列: P3(10) P4(20)

Time 31: 运行进程名 P3 (剩余 10->9)
        队列: P3(9) P4(20)

Time 32: 运行进程名 P3 (剩余 9->8)
        队列: P3(8) P4(20)

Time 33: 运行进程名 P3 (剩余 8->7)
```

```
        队列: P3(7) P4(20)

Time 34: 运行进程名 P3 (剩余 7->6)
        队列: P3(6) P4(20)

Time 35: 运行进程名 P3 (剩余 6->5)
        队列: P3(5) P4(20)

Time 36: 运行进程名 P3 (剩余 5->4)
        队列: P3(4) P4(20)

Time 37: 运行进程名 P3 (剩余 4->3)
        队列: P3(3) P4(20)

Time 38: 运行进程名 P3 (剩余 3->2)
        队列: P3(2) P4(20)

Time 39: 运行进程名 P3 (剩余 2->1)
        队列: P3(1) P4(20)

Time 40: 运行进程名 P3 (剩余 1->0)  <进程 P3 已完成, 踢出队列>
        队列: P4(20)

Time 41: 运行进程名 P4 (剩余 20->19)
        队列: P4(19)

Time 42: 运行进程名 P4 (剩余 19->18)
        队列: P4(18)

Time 43: 运行进程名 P4 (剩余 18->17)
        队列: P4(17)

Time 44: 运行进程名 P4 (剩余 17->16)
        队列: P4(16)

Time 45: 运行进程名 P4 (剩余 16->15)
        队列: P4(15)

Time 46: 运行进程名 P4 (剩余 15->14)
        队列: P4(14)

Time 47: 运行进程名 P4 (剩余 14->13)
        队列: P4(13)

Time 48: 运行进程名 P4 (剩余 13->12)
        队列: P4(12)

Time 49: 运行进程名 P4 (剩余 12->11)
        队列: P4(11)

Time 50: 运行进程名 P4 (剩余 11->10)
        队列: P4(10)

Time 51: 运行进程名 P4 (剩余 10->9)
        队列: P4(9)

Time 52: 运行进程名 P4 (剩余 9->8)
        队列: P4(8)

Time 53: 运行进程名 P4 (剩余 8->7)
        队列: P4(7)

Time 54: 运行进程名 P4 (剩余 7->6)
        队列: P4(6)

Time 55: 运行进程名 P4 (剩余 6->5)
        队列: P4(5)

Time 56: 运行进程名 P4 (剩余 5->4)
        队列: P4(4)

Time 57: 运行进程名 P4 (剩余 4->3)
        队列: P4(3)

Time 58: 运行进程名 P4 (剩余 3->2)
        队列: P4(2)

Time 59: 运行进程名 P4 (剩余 2->1)
        队列: P4(1)

Time 60: 运行进程名 P4 (剩余 1->0)  <进程 P4 已完成, 踢出队列>
        队列: 空

所有进程调度完成, 总时间 = 61
```

五、实验结果及分析

实验结果为成功，实验要求被完全实现。

抢占式 SJF 算法对比 Round Robin 算法，所有进程总开销时间一致，但进程平均完成时间更低。但在操作系统中，实际上无法实现 SJF 算法，因为无法提前知道程式需要运行时间是多少。

通过本次实验，我获得以下几点收获：

1. 加深对操作系统核心概念的理解。
2. 掌握调度算法：抢占式 SJF 算法和 Round Robin 算法的实现细节。
3. 锻炼了 C 语言指针与动态数据结构编程能力。
4. 熟悉 malloc/free 或静态数组等不同方式，了解到了内存分配与释放对程序健壮性的影响。

备注：将代码和实验报告打包发邮件

```
#include <iostream>
#include <string>
#include <vector>
#include <limits>
#include <algorithm>
using namespace std;

struct PCB { // 进程控制块结构体
    string name; // 进程名称
    int arrive_time; // 到达时间
    int burst_time; // 要求运行时间
    int remaining_time; // 剩余运行时间
    PCB* next; // 队列指针，指向下一个 PCB（仅用于 RR 调度）
};

void sort_by_arrival(PCB* pcs, int N) {
    // 将阵列按到达时间排序
    for (int i = 0; i < N - 1; ++i) {
        int min_idx = i;
        for (int j = i + 1; j < N; ++j) {
            if (pcs[j].arrive_time < pcs[min_idx].arrive_time)
                min_idx = j;
        }
        if (min_idx != i) {
            swap(pcs[i], pcs[min_idx]); // 交换位置
        }
    }
}

void enqueue(PCB*& head, PCB*& tail, PCB* p) {
    // 就绪队列入队操作（RR）
    p->next = nullptr; // 新节点指向空
    if (!head) head = p; // 判断队列为空，如是，设置传进来的节点为头节点
    else tail->next = p; // 否则将传进来的节点链接到队尾
    tail = p; // 更新队尾指针
}
```

```
}
```

```
PCB* dequeue(PCB*& head, PCB*& tail) {  
    // 就绪队列入队操作 (RR)  
    if (!head) return nullptr; //判断队列是否为空, 如空返回 nullptr  
    PCB* p = head;//取出队首  
    head = head->next;//更新队首  
    if (!head) tail = nullptr; // 队列空则更新队尾  
    p->next = nullptr; // 断开链接  
    return p;  
}
```

```
void dd_rr(PCB* pcs, int N) {  
    // 抢占式 Round Robin 调度实现  
    PCB* head = nullptr;  
    PCB* tail = nullptr;  
    int time = 0, completed = 0, next_idx = 0; // 初始化变量: 当前时间、完成的进程数、下一个未入队进程的索引  
    //start 初始时将所有已到达进程入队  
    while (next_idx < N && pcs[next_idx].arrive_time <= time) {  
        enqueue(head, tail, &pcs[next_idx]);  
        ++next_idx;  
    }  
    //end 初始时将所有已到达进程入队  
  
    cout << "\n 使用 Round Robin 调度算法\n";  
    while (completed < N) { // 循环直到所有进程完成  
        if (!head) { // 判断队列是否为空, 如是 CPU 空闲  
            cout << "Time " << time << ": CPU 空闲\n\n";  
            ++time;  
            // 检查新到达进程  
            while (next_idx < N && pcs[next_idx].arrive_time <= time) {  
                enqueue(head, tail, &pcs[next_idx]);  
                ++next_idx;  
            }  
            continue;  
        }  
        // 出队一个进程执行一个时间片  
        PCB* cur = dequeue(head, tail);  
        cout << "Time " << time << ": 运行进程名 " << cur->name;  
        --(cur->remaining_time);// 剩余时间减 1  
        ++time; //时间前进  
        cout << " (剩余 " << cur->remaining_time + 1 << "->" << cur->remaining_time << " )";  
        // 检查当前时间点的新到达进程  
        while (next_idx < N && pcs[next_idx].arrive_time <= time) {  
            enqueue(head, tail, &pcs[next_idx]);  
            ++next_idx;  
        }  
    }  
}
```

```

// 若当前进程尚未完成，重新入队
if (cur->remaining_time > 0) enqueue(head, tail, cur);
else {
    // 进程完成，从队列中移除
    cout << "\t<进程 " << cur->name << " 已完成，踢出队列>";
    ++completed;
}
if (!head) {
    cout << "\n\t\t 队列: 空\n";
} else {
    cout << "\n\t\t 队列: ";
    for (PCB* p = head; p; p = p->next) {
        cout << p->name << "(剩余 " << p->remaining_time << ")\t";
    }
}
cout << endl<<endl;
}
cout << "所有进程调度完成，总时间 = " << time << endl;
}

```

```

void dd_sjf(PCB* pcs, int N) {
    // 抢占式 SJF 调度实现
    vector<PCB*> ready; // 初始化就绪队列变量
    int time = 0, completed = 0, next_idx = 0; // 初始化变量：系统当前时间、已完成进程计数、下一个待入队进程
索引
    cout << "\n 使用 抢占式短作业优先 调度算法\n";

```

```

while (completed < N) { // 主循环：直到所有进程完成

```

```

    // start 把到达的进程加入 ready
    while (next_idx < N && pcs[next_idx].arrive_time <= time) {
        ready.push_back(&pcs[next_idx]);
        ++next_idx;
    }
    // end 把到达的进程加入 ready

```

```

    if (ready.empty()) { // 判断 ready 是否为空，如是则 CPU 空闲
        cout << "Time " << time << ": CPU 空闲\n\n";
        ++time;
        continue;
    }

```

```

    // 选剩余时间最短

```

```

    auto cmp = [](PCB* a, PCB* b) { return a->remaining_time < b->remaining_time; }; // 定义比较函数：返回
true 表示 a 的剩余时间小于 b

```

```

    auto it = min_element(ready.begin(), ready.end(), cmp); // 找到 ready 中最短进程的迭代器

```

```

    PCB* cur = *it; // 解引用迭代器得到当前选中的进程

```

```

    cout << "Time " << time << ": 运行进程名 " << cur->name; // 打印当前时间和进程名称，表示开始执行
    --(cur->remaining_time); // 执行一个时间单位，进程剩余时间减 1

```

```

        ++time;// 时间+1
        cout << " (剩余 " << cur->remaining_time +1<< "->"<<cur->remaining_time << ")";
        // 完成检测
        if (cur->remaining_time == 0) { // 判断进程是否完成，如是，移除并计数
            cout << "\t进程 " << cur->name << " 已完成，踢出队列>";
            ready.erase(it);
            ++completed;
        }

        if (ready.empty()) cout << "\n\t\t队列: 空" << endl;
        else {
            cout << "\n\t\t队列: ";
            for (auto p : ready) cout << p->name << "(" << p->remaining_time << ") ";
        }

        cout << endl<< endl;
    }
    cout << "所有进程调度完成，总时间 = " << time << endl;
}

int main() {
    int N, dd_choice;
    cout << "\n 选择调度算法 (1: Round Robin; 2: 抢占式 SJF): ";
    cin >> dd_choice;
    cout << "请输入进程数量 (N>=4): ";
    while (!(cin >> N) || N < 4) {
        cout << "输入无效，请输入 N>=4 的整数: ";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
    PCB* pcs = new PCB[N]; // 动态分配 PCB 阵列
    for (int i = 0; i < N; ++i) {
        cout << "\n 进程 " << i+1 << " 名称: "; cin >> pcs[i].name;
        cout << "进程 " << i+1 << " 到达时间: "; cin >> pcs[i].arrive_time;
        cout << "进程 " << i+1 << " 要求运行时间: "; cin >> pcs[i].burst_time;
        pcs[i].remaining_time = pcs[i].burst_time;
        pcs[i].next = nullptr;
    }
    sort_by_arrival(pcs, N); //按到达时间排序

    if (dd_choice == 1) dd_rr(pcs, N);
    else if (dd_choice == 2) dd_sjf(pcs, N);
    else cout << "无效选择，程序结束" << endl;
    delete[] pcs; // 释放内存
    return 0;
}

```